

OOP Learning Roadmap

We'll go in this order:

1. OOPs style and properties
2. Classes & Objects → the foundation
3. Attributes & Methods → properties and behaviors
4. Constructor (`__init__`) and `self` keyword
5. Encapsulation → data hiding and access control
6. Inheritance → reusing and extending functionality
7. Polymorphism → one interface, many forms
8. Abstraction → hiding complex implementation
9. Class vs Instance Variables & Methods
10. Magic / Dunder Methods (like `__str__`, `__len__`, etc.)
11. Real-World OOP Project (like a Banking System, Student Management, or E-commerce Cart)
12. Practice Exercises

What Are Programming Paradigms?

A **programming paradigm** is basically a *style or approach* to writing and organizing code — a way of thinking about how to solve problems.

Think of paradigms as “different mindsets”:

- **Procedural** → *Step-by-step instructions (how to do it)*
- **Object-Oriented** → *Represent real-world things (what it is)*
- **Functional** → *Describe transformations (what needs to be done)*

Each paradigm solves problems in a different way, and modern languages (like Python, JavaScript, etc.) often support **multiple paradigms**.

1. Procedural Programming

Concept:

It's all about writing *procedures* (or functions) that operate on data.
You break down a big task into smaller steps and execute them in order.

Think of it like: A recipe — a set of instructions you follow in sequence.

Example (Python):

```
# Procedural approach
def calculate_area(radius):
    return 3.14 * radius * radius

def display_area(radius):
    area = calculate_area(radius)
    print(f"Area of circle: {area}")

radius = 5
display_area(radius)
```

Use Cases:

- Simple scripts

- Small tools or utilities
- Data processing or automation tasks

Advantages:

- Easy to start with
- Great for small programs
- Direct and straightforward

Disadvantages:

- Hard to maintain as the program grows
- Data and logic are *separate* (can lead to tight coupling)
- Difficult to model real-world entities

2. Object-Oriented Programming (OOP)

Concept:

Focuses on *objects* — which bundle **data (attributes)** and **behavior (methods)** together.

You model real-world entities like “Car,” “Customer,” or “Employee” as objects with properties and actions.

Think of it like: You’re building mini-worlds of interacting objects.

Example (Python):

```
# Object-Oriented approach
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

circle1 = Circle(5)
```

```
print("Area of circle:", circle1.area())
```

Use Cases:

- Large applications (banking, e-commerce, games)
- Systems with many entities interacting (CRM, ERPs)
- When you need reusability, scalability, or abstraction

Advantages:

- Easier to maintain and extend (via inheritance, polymorphism)
- Models real-world entities naturally
- Code reusability through classes
- Encapsulation improves data safety

Disadvantages:

- More complex to design initially
- Might be overkill for small scripts

3. Functional Programming

Concept:

Treats computation as *evaluations of pure functions* — no changing state, no side effects.

Think of it like: You give inputs, you get outputs. That's it. No modifying variables in between.

Example (Python):

```
# Functional approach
from math import pi

area = lambda r: pi * r * r
print("Area of circle:", area(5))
```

Or using **map**, **filter**, **reduce**:

```
numbers = [1, 2, 3, 4]
squares = list(map(lambda x: x**2, numbers))
print(squares) # [1, 4, 9, 16]
```

Use Cases:

- Data transformations (ETL, analytics pipelines)
- Parallel or distributed computing
- When predictability and immutability matter (e.g. finance, AI)

Advantages:

- Easier to test and debug (no side effects)
- Short, expressive code
- Great for concurrent programming

Disadvantages:

- Can be abstract/hard to understand for beginners
- Doesn't model real-world objects well

Major Differences: OOP vs Procedural vs Functional

Feature	Procedural	Object-Oriented (OOP)	Functional
Focus	Functions & procedures	Objects (data + behavior)	Pure functions & data transformations
Data Handling	Separate from functions	Bundled with objects	Immutable

Code Reuse	Via functions	Via inheritance & polymorphism	Via composition of functions
State Management	Global or local variables	Encapsulated in objects	Avoided (no mutable state)
Real-world Modeling	Not natural	Very natural	Not focused on real-world entities
Ease of Testing	Moderate	Moderate	Easy (pure functions)
Use Case	Simple scripts	Large systems	Data transformations

So Why Use OOP If Procedural Can Do the Same?

You're right — **you *can* achieve almost anything procedurally.**
But OOP is not about *capability*, it's about *manageability*.

Let's see a simple analogy

Example: Building a Car Management System

Procedural way:

```
car_name = "Tesla"
car_speed = 120

def accelerate(speed):
    return speed + 10

car_speed = accelerate(car_speed)
```

If you now add 100 cars — it becomes chaos with multiple variables and functions managing each one.

OOP way:

```
class Car:
    def __init__(self, name, speed):
        self.name = name
        self.speed = speed

    def accelerate(self):
```

```

        self.speed += 10

car1 = Car("Tesla", 120)
car2 = Car("BMW", 100)

car1.accelerate()
print(car1.speed)

```

Now each car carries its own data and methods, making it easy to manage and extend.

OOP shines in scalability and abstraction, not just functionality.

When to Use What?

Situation	Best Paradigm	Why
Small automation script	Procedural	Quick, simple, readable
Large software system (e.g., ecommerce app, game engine)	OOP	Scalable, modular, maintainable
Data transformation pipeline or ML preprocessing	Functional	Concise, stateless, easier to parallelize
API or Web framework	OOP (with some functional)	Classes to model entities, but functions to handle logic

Summary

Paradigm	Mindset	Example
Procedural	"How do I do it step by step?"	Cooking recipe
OOP	"What entities exist and how do they interact?"	Managing cars, customers
Functional		Converting, mapping, reducing data

Topic 1: Classes and Objects

Let's start from the base.

In Python (and in OOP generally):

- A **Class** is a *blueprint* for creating objects.
- An **Object** is an *instance* of a class — a real thing built from that blueprint.

Think of:

- **Class** → “Car blueprint”
- **Object** → “Tesla Model 3”, “BMW i8”

Each object can have:

- **Attributes (variables)** → define properties (e.g., **color**, **speed**)
- **Methods (functions)** → define behavior (e.g., **drive()**, **brake()**)

Example 1: The Simplest Class

```
class Car:  
    pass # an empty class for now
```

```
# Creating objects
```

```
car1 = Car()
```

```
car2 = Car()
```

```
print(car1)
```

```
print(car2)
```

```
# x = 5
```

This just creates two objects from the same blueprint.

Even though they're empty, Python still knows they belong to the **Car** class.

Example 2: Adding Attributes & Methods

```
class Car:
    def __init__(self, b, colour):
        self.brand = b
        self.color = colour

    def start_engine(self):
        print(f"{self.brand} is now running!")

# Creating objects
car1 = Car("Tesla", "Red")
car2 = Car("BMW", "Blue")

# Accessing attributes
print(car1.brand) # Tesla
print(car2.color) # Blue

# Calling method
car1.start_engine() # Tesla is now running!
```

Key Points

Concept	Description
Class	A blueprint or template for creating objects
Object	An instance created from the class
<code>__init__</code> method	
<code>self</code>	
Attributes	Data associated with the object
Methods	Functions that operate on object data
	Constructor — called automatically when you create an object
	Refers to the current instance (object)

Use Case: Modeling a Real-world Entity

Let's say you're working in a **ride-sharing company (like Uber)**.
You might represent each driver as an object.

```
class Driver:
    def __init__(self, name, rating):
        self.name = name
        self.rating = rating

    def accept_ride(self):
        print(f"{self.name} has accepted the ride!")

    def update_rating(self, new_rating):
        self.rating = new_rating
        print(f"{self.name}'s new rating: {self.rating}")

# Using it
driver1 = Driver("John", 4.8)
driver2 = Driver("Emma", 4.5)

driver1.accept_ride()
driver2.update_rating(4.7)
```

Here:

- Each driver is an **object**
- They share the same **class structure**
- Each can have unique **data and actions**

Practice Questions:

Let's test what you've learned before moving forward:

Q1. What's the difference between a *class* and an *object*?

Q2. Why do we use `self` in class methods?

Q3. What does the `__init__` method do?

Q4. Write a small class called `Book` that has attributes: `title`, `author`, and `price`.

- Add a method `display_info()` that prints these details.
- Then, create two objects of this class and call the method.

Static/ Class Variables and Instance Variables in Python

We already know about Class and Instance in OOPs

Example:

```
class Car:
```

```
    pass
```

```
c1 = Car()
```

```
c2 = Car()
```

- `Car` → class
- `c1`, `c2` → instances (individual objects of the class)

2. Types of Variables in OOP

There are mainly **three** types of variables in a class:

Type	Defined Where	Belongs To	Shared Between Objects?
Instance variable	Inside methods, using <code>self</code>	Individual object	 No

Class variable	Inside class but outside methods	The class itself	✅ Yes
Static variable / attribute	Similar to class variables, accessed using class name	Class level	✅ Yes (often constants)

A. Instance Variables:

Variables that are unique to **each object** (instance).

They are defined using **self** inside the `__init__` method or any instance method.

Example:

```
class Car:

    def __init__(self, brand, color):

        self.brand = brand    # instance variable

        self.color = color    # instance variable

c1 = Car("Tesla", "Red")

c2 = Car("BMW", "Black")

print(c1.brand) # Tesla

print(c2.brand) # BMW
```

Intuition:

- Each car object has **its own copy** of **brand** and **color**.
- Changing **c1.color** won't affect **c2.color**.

- Instance variables = **object's personal data**.

B. Class Variables

Definition:

Variables that are shared **by all objects** of a class.

They are defined directly **inside the class**, not inside any method.

Example:

```
class Car:
```

```
    wheels = 4  # class variable (common for all)
```

```
    def __init__(self, brand, color):
```

```
        self.brand = brand
```

```
        self.color = color
```

```
c1 = Car("Tesla", "Red")
```

```
c2 = Car("BMW", "Black")
```

```
print(c1.wheels) # 4
```

```
print(c2.wheels) # 4
```

```
Car.wheels = 6
```

```
print(c1.wheels) # 6
```

```
print(c2.wheels) # 6
```

Intuition:

- Class variables are **shared memory** across all objects.
- If you change it through the class name, the change is visible everywhere.
- Example in real life: "All cars have 4 wheels" → this property belongs to the **Car class**, not each specific car.

C. Static Variables (Alternate View)

In Python, **static variables** are not separate from class variables — but the term "static" is often used when the variable:

- Doesn't depend on class or instance behavior
- Acts like a constant or configuration

Example:

```
class MathTools:
```

```
    PI = 3.14159 # static/class variable
```

```
    @staticmethod
```

```
    def circle_area(radius):
```

```
        return MathTools.PI * radius ** 2
```

Here:

- **PI** is a **static/class variable**
- It's used by the class, not by individual instances.

3. How They Differ in Memory and Behavior

Feature	Instance Variable	Class Variable
Defined inside	<code>__init__</code> or instance method (using <code>self</code>)	Class directly (outside methods)

Belongs to	Object	Class
Memory	Separate copy for each instance	One copy shared by all
Accessed via	<code>self.varname</code>	<code>ClassName.varname</code> or <code>self.varname</code>
Modified via	Only affects that object	Affects all objects if modified via class name

Example Combining All Three

```
class Employee:
```

```
    company_name = "TechCorp" # class variable
```

```
    def __init__(self, name, salary):
```

```
        self.name = name    # instance variable
```

```
        self.salary = salary # instance variable
```

```
    @staticmethod
```

```
    def company_policy():
```

```
        return "Work from home allowed 2 days/week" # static method
```

```
e1 = Employee("Akshat", 70000)
```

```
e2 = Employee("Riya", 80000)
```

```
print(e1.company_name) # TechCorp
```

```
print(e2.company_name) # TechCorp
```

```
print(e1.name)        # Akshat
```

```
print(e2.name)        # Riya
```

```
Employee.company_name = "DataWorks"
```

```
print(e1.company_name) # DataWorks
```

```
print(e2.company_name) # DataWorks
```

`name` and `salary` → different for each employee (instance vars)

`company_name` → shared across all (class var)

`company_policy()` → static method (belongs to the class)

Deep Intuition (Real-World Analogy)

Imagine you're designing a system for a **school**:

Concept	Example	OOP Equivalent
The school blueprint	Defines what every student must have	<code>class Student:</code>
Each student	Has their own name, roll number, marks	Instance variables

School rules	Common for all students	Class / static variables
A student's name change	Affects only that student	Instance variable update
Change in school name	Affects all students	Class variable update

Another example: Instance vs Class Variable Access

```
class Demo:
```

```
    x = 10  # class variable
```

```
    def __init__(self):
```

```
        self.y = 20  # instance variable
```

```
d1 = Demo()
```

```
d2 = Demo()
```

```
d1.x = 100  # creates new instance variable for d1 only
```

```
Demo.x = 200 # changes class variable for all
```

```
print(d1.x) # 100 (instance var shadows class var)
```

```
print(d2.x) # 200
```

Important insight:

If you assign to `self.x`, Python **creates a new instance variable**, shadowing the class variable for that object.

Summary

Type	Defined Where	Shared?	Accessed By	Example
Instance Variable	Inside <code>__init__</code> using <code>self</code>	 No	<code>obj.var</code>	<code>self.name</code>
Class Variable	Inside class (outside methods)	 Yes	<code>Class.var</code> or <code>obj.var</code>	<code>Car.wheels</code>
Static Variable	Same as class var, used with constants	 Yes	<code>Class.var</code>	<code>MathTools.PI</code>

Method Overloading vs Method Overriding

Method Overriding (OOP concept)

Definition:

When a subclass defines a method with the same name as a method in its parent class, and the subclass version replaces (or overrides) the parent version.

In short —

Same method name

Same parameters

Different behavior (in child class)

Example: Method Overriding

```
class Animal:
    def speak(self):
```

```
        print("Animal makes a sound")

class Dog(Animal):

    def speak(self):

        print("Dog barks")

# Example

a = Animal()

a.speak()      # Animal makes a sound


d = Dog()

d.speak()      # Dog barks  <-- overridden method
```

Here, **Dog** overrides the **speak()** method of **Animal**.

When you call **speak()** on a **Dog** object, Python uses the child class version, not the parent's.

Use case:

You override methods to provide specialized behavior for subclasses while keeping a consistent interface.

2. Method Overloading

Definition:

When multiple methods have the same name but different parameters or argument types.

In many languages like Java or C++, this is allowed (compile-time polymorphism).

But in Python, true method overloading doesn't exist — because the last defined method with the same name overwrites the previous ones.

However, Python can simulate overloading using:

- Default arguments
- Variable-length arguments (`*args`, `**kwargs`)

Example: Simulated Method Overloading

```
class MathOps:

    def add(self, a=None, b=None, c=None):

        if a is not None and b is not None and c is not
None:

            return a + b + c

        elif a is not None and b is not None:

            return a + b

        else:

            return a

m = MathOps()

print(m.add(2, 3))          # 5
```

```
print(m.add(2, 3, 4))      # 9
```

```
print(m.add(5))           # 5
```

Same method name: **add**

Different behavior depending on number of parameters

Simulated using optional arguments

3. Operator Overloading

Definition:

*It's a special type of method overloading where operators like **+**, **-**, **>**, etc. are made to work with user-defined objects.*

This uses magic (dunder) methods like **__add__**, **__sub__**, etc.

Example: Operator Overloading

```
class Student:
```

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def __add__(self, other):
```

```
        return self.marks + other.marks
```

```
s1 = Student("Akshat", 85)
```

```
s2 = Student("Riya", 90)
```

```
print(s1 + s2)    # Output: 175
```

Here, the **+** operator calls the magic method `__add__()`.

So you've overloaded the **+** operator to add marks instead of normal numbers.

Operator Overloading in Python OOPs:

1. How Python Normally Handles Operators

When you write something like:

```
x = 5 + 10
```

You're actually not doing something *magical*.

Behind the scenes, Python translates that **+** into a **method call**:

```
x = int.__add__(5, 10)
```

Here:

- `5` is an instance of the `int` class.
- The **+** operator calls the **special method** `__add__()` that's built into the `int` class.
- So effectively, `5 + 10 = int.__add__(5, 10) → 15`.

Every data type in Python has its own implementation of such operator methods — for example:

- `__sub__` → subtraction

- `__mul__` → multiplication
- `__gt__` → greater than (>)
- and many more.

2. So What Happens With a Custom Class?

Let's say you create your own class:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
```

Now, what if you try:

```
s1 = Student("Akshat", 85)
s2 = Student("Riya", 90)

print(s1 + s2)
```

Python will throw an error:

`TypeError: unsupported operand type(s) for +: 'Student' and 'Student'`

Why?

Because the `Student` class **doesn't know what "+" means** for two student objects.

Unlike integers or strings, your custom object has no built-in rule for addition.

3. What "Overloading" Really Does

Now you tell Python *what +* should mean for `Student` objects by **defining a special method** — `__add__`.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
```

```
self.marks = marks
```

```
def __add__(self, other):  
    return self.marks + other.marks
```

Now, when you write:

```
print(s1 + s2)
```

Python internally converts it into:

```
print(s1.__add__(s2))
```

And since you've defined what `__add__` means, Python now knows how to perform the operation.

The Fundamental Idea: When you “overload” an operator, you are **telling Python how to interpret that operator** when used with your own custom objects.

So here:

- Normally `+` adds numbers (`int.__add__`),
- But **you** decided that for `Student`, `+` should **add their marks**.

In essence:

You are giving *new meaning* to the existing `+` operator — specifically for your class.

That's why it's called operator overloading:

- You're overloading (redefining) the meaning of the operator
- Based on the **type of operands** involved

4. Deep Intuition

When Python sees `a + b`, it does this:

1. Check the **class** of `a`
2. Look for a method called `__add__` in that class
3. Call it like `a.__add__(b)`
4. If it doesn't exist, look for `b.__radd__(a)` (reverse add)
5. If neither exists → throw a `TypeError`

So if you think fundamentally:

- Operators like `+`, `-`, `>`, `in` etc. are just **syntactic sugar**.
- They map directly to **methods** that define behavior for that operator.

5. You Can Define Any Behavior You Want

Example —

Let's make `+` return a **new student** combining both names and average marks:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def __add__(self, other):
        new_name = self.name + " & " + other.name
        avg_marks = (self.marks + other.marks) / 2
        return Student(new_name, avg_marks)

    def __str__(self):
        return f"{self.name} -> {self.marks}"

s1 = Student("Akshat", 85)
s2 = Student("Riya", 95)
s3 = s1 + s2

print(s3) # Akshat & Riya -> 90.0
```

Now the `+` operator **creates a new combined object** — that's your custom definition of addition.

Summary in One Line:

Operator Overloading = Teaching Python how to use its existing operators on your custom objects.

Encapsulation in OOPs:

Encapsulation means *“wrapping up data and the methods that operate on that data into a single unit”* — i.e., a class.

It helps you protect the data from accidental modification and control how it's accessed.

In simple words:

“Encapsulation = Data hiding + Controlled access.”

Intuition behind Encapsulation:

Think of encapsulation like how a bank account works. You can deposit or withdraw money through the bank's official methods, but you can't just walk in and directly edit your account balance in their database. The actual balance is hidden and protected inside the system, and you can only access it through predefined actions like *deposit()* or *withdraw()*. This ensures your money stays consistent and no one accidentally breaks the rules. Similarly, in programming, encapsulation means keeping the data (like **balance**) safe inside a class and allowing access only through controlled functions (methods). It's like putting a lock on your data and giving users a proper key instead of letting them mess with the machinery inside.

Key Concepts Behind Encapsulation

Concept

Description

Public Members Can be accessed anywhere.

Protected Members Should not be accessed outside the class (convention).

Private Members Strictly hidden and can't be accessed directly outside the class.

Access Modifiers in Python

Python doesn't have true private variables like Java or C++. Instead, it follows naming conventions:

Access Type	Syntax	Example	Accessible From
Public	Normal name	<code>self.name</code>	Anywhere
Protected	One underscore —	<code>self._salary</code>	Inside class & subclasses (by convention)
Private	Two underscores __	<code>self.__balance</code>	Only inside the class (name mangling)

Example 1: Basic Encapsulation

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name      # public
```

```
        self._dept = "Finance" # protected
```

```
        self.__salary = salary # private
```

```
    def show_info(self):
```

```
        print(f"Employee: {self.name}, Dept: {self._dept}")
```

```
    def get_salary(self):
```

```
        return self.__salary
```

```
    def set_salary(self, amount):
```

```
        if amount > 0:
```

```
            self.__salary = amount
```

```
        else:
```

```
            print("Invalid salary")
```

```
emp = Employee("Akshat", 50000)
```

```
print(emp.name)      #  Accessible
```

```
print(emp._dept)     #  Accessible, but not recommended
```

```
print(emp.get_salary()) # ✓ Access via method
```

```
emp.set_salary(60000)
```

```
print(emp.get_salary()) # Updated salary
```

What's happening:

- `__salary` is private → can't access directly like `emp.__salary`.
- We use getter and setter methods (`get_salary`, `set_salary`) for controlled access.
- That's the *core of encapsulation* — we're hiding data and exposing it safely.

Example 2: Encapsulation with Validation

```
class BankAccount:
```

```
    def __init__(self, owner, balance):
```

```
        self.owner = owner
```

```
        self.__balance = balance # Private variable
```

```
    def deposit(self, amount):
```

```
        if amount > 0:
```

```
            self.__balance += amount
```

```
        else:
```

```
            print("Deposit must be positive!")
```

```
def withdraw(self, amount):  
    if 0 < amount <= self.__balance:  
        self.__balance -= amount  
    else:  
        print("Insufficient funds or invalid amount!")
```

```
def get_balance(self):  
    return self.__balance
```

Usage

```
account = BankAccount("Riya", 1000)  
account.deposit(500)  
print(account.get_balance()) #1500  
account.withdraw(2000)      # Insufficient funds
```

Here, balance is hidden to prevent misuse (like `account.__balance = -500`).

Why Encapsulation Matters

Data Protection: Prevents accidental or unauthorized modification.

Controlled Access: You decide how data is read or written.

Easier Maintenance: Changes inside a class don't break the outer code.

Better Security & Abstraction: Hides implementation details from users.

Real-Life Applications

Domain	Example of Encapsulation
Banking	Account balance hidden; only deposit/withdraw methods control changes.
E-commerce	Product stock updated only via internal methods.
Healthcare	Patient medical history accessible only via authorized functions.
Software APIs	Internal logic hidden, only exposing needed methods (like <code>.fit()</code> in scikit-learn).

Getters & Setters: Methods used to safely access and modify private data.

Quick Demo: Accessing Private Variables with Name Mangling

```
class Demo:
```

```
    def __init__(self):
```

```
        self.__secret = "hidden"
```

```
obj = Demo()
```

```
# print(obj.__secret)  # ❌ Error
```

```
print(obj._Demo__secret) # Works but not recommended
```

Objectname.__classname__variablename -> this is how the private variables are saved in the memory.

“Encapsulation is about keeping data safe and interactions clean — like giving someone a TV remote instead of letting them open the TV circuit.”

Reference Variables & Pass by reference in Python:

1. What is a Reference Variable?

In Python, a reference variable is just a name that refers to an object stored in memory.

When you do:

```
x = [1, 2, 3]
```

- The list `[1, 2, 3]` is created in memory.
- The variable `x` doesn't *store* the list — it just points (refers) to it.

If you now do:

```
y = x
```

Both `x` and `y` point to the same list in memory — there is still only *one list*, but now *two names* for it.

```
x = [1, 2, 3]
```

```
y = x
```

```
y.append(4)
```

```
print(x) # [1, 2, 3, 4]
```

```
print(y) # [1, 2, 3, 4]
```

Both `x` and `y` show `[1, 2, 3, 4]`

Because they refer to the same object in memory — changing one changes the other.

2. Pass by Reference (in Functions)

When you pass a variable (like a list or object) to a function, Python passes the reference, not a copy of the object.

That means the function can modify the original object.

Example

```
def modify_list(lst):  
    lst.append(100)  
    print("Inside function:", lst)
```

```
nums = [1, 2, 3]  
modify_list(nums)  
print("Outside function:", nums)
```

Output:

Inside function: [1, 2, 3, 100]

Outside function: [1, 2, 3, 100]

Because `lst` and `nums` are references to the same list — so the function works on the same memory location.

3. Pass by Value (for Immutable Types)

If you pass immutable types (like int, float, string, tuple), they don't change, because Python creates a *new object* when you modify them.

Example:

```
def change_number(n):
```

```
n = n + 10
```

```
print("Inside function:", n)
```

```
x = 5
```

```
change_number(x)
```

```
print("Outside function:", x)
```

Output:

Inside function: 15

Outside function: 5

Here, integers are immutable, so changing **n** inside the function creates a new object, leaving the original **x** unchanged.

4. Real-Life Analogy

Think of a reference like sharing a Google Doc link:

- When you share the *link*, everyone edits the same document (like passing by reference).
- When you send a *PDF copy*, everyone edits their own separate version (like pass by value).

In Python:

- Lists, dicts, sets, and class objects → behave like Google Docs (mutable, passed by reference)
- Integers, strings, tuples → behave like PDFs (immutable, copied on modification)

Summary Table

Type	Mutable?	Passed By	Effect
int, float, str, tuple	Immutable	Value (copy created when changed)	Original not affected
list, dict, set, object	Mutable	Reference	Changes affect original

Quick Real Example (with class objects)

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
def rename(person):
```

```
    person.name = "Rahul"
```

```
p1 = Person("Akshat")
```

```
rename(p1)
```

```
print(p1.name)
```

Output: Rahul

Here, the object **p1** is passed by reference, so changing **person.name** inside the function directly updates **p1**.

Inheritance in OOPs Python:

Inheritance is the mechanism in Object-Oriented Programming (OOP) that allows one class (called the *child* or *subclass*) to acquire the properties and behaviors (variables and methods) of another class (called the *parent* or *base class*).

Think of it as "reusing and extending existing code".

Intuition — Real World Analogy

Imagine a "Parent Class" as a *template of characteristics and behaviors*.

Example:

- A **Human** has features like **name**, **age**, and behaviors like **speak()** and **walk()**.
- Now, a **Student** is also a human, but with *some extra abilities*, like **study()** or **take_exam()**.

So instead of rewriting everything from scratch for the **Student** class, we can just inherit from the **Human** class.

That's inheritance in action.

1. What is Inheritance?

Inheritance is an Object-Oriented Programming (OOP) feature that allows one class (called a child class) to acquire properties and methods from another class (called a parent class).

It enables:

- **Code Reusability**
- **Extensibility**

- Logical Hierarchies
- Reduced Redundancy

Real-life Analogy:

A child inherits qualities (like eye color, surname, traditions) from parents but can also have its own traits.

Similarly, a class can inherit data and behavior from another class while adding its own.

Example:

```
class Parent:
```

```
    def show(self):
```

```
        print("This is the parent class")
```

```
class Child(Parent):
```

```
    def display(self):
```

```
        print("This is the child class")
```

```
c = Child()
```

```
c.show()    # inherited method
```

```
c.display() # own method
```

Output:

```
This is the parent class
```

This is the child class

The **Child** didn't define **show()** — it inherited it from **Parent**.

2. Why Inheritance?

Key Goals:

1. **Code Reusability:** Use existing code instead of rewriting.
2. **Extensibility:** Extend functionality easily.
3. **Maintainability:** Fix/update once, reflect everywhere.
4. **Modeling Real Relationships:** Represent real-world hierarchies (like Company → Manager → Employee).

Example (Reusability in Action)

```
class Vehicle:
```

```
    def start(self):
```

```
        print("Starting engine...")
```

```
class Car(Vehicle):
```

```
    def drive(self):
```

```
        print("Car is driving...")
```

```
c = Car()
```

```
c.start() # From Vehicle
```

```
c.drive() # Own method
```

Car automatically gets `start()` from `Vehicle`.

3. The “IS-A” Relationship

Inheritance represents an “is-a” relationship.

- A `Car` is a `Vehicle`
- A `Dog` is an `Animal`
- A `Car` is not an `Engine` → that’s a “*has-a*” relationship → use Composition, not inheritance.

4. Types of Inheritance in Python

Type	Structure	Meaning
Single	1 parent → 1 child	Basic one-to-one
Multiple	multiple parents → 1 child	Inherits from many
Multilevel	parent → child → grandchild	Inheritance chain

Hierarchical	1 parent → multiple children	Shared base
Hybrid	combination of above	Complex real-world scenarios

Example: All in One Place

(1) Single Inheritance

```
class Animal:
```

```
    def speak(self):  
        print("Animal speaks")
```

```
class Dog(Animal):
```

```
    def bark(self):  
        print("Dog barks")
```

```
d = Dog()
```

```
d.speak()
```

```
d.bark()
```

(2) Multiple Inheritance

```
class Father:
```

```
    def skills(self):  
        print("Can drive")
```



```
class Mother:
```

```
    def skills(self):
```

```
        print("Can cook")
```

```
class Child(Father, Mother):
```

```
    def skills(self):
```

```
        super().skills()
```

```
        print("Can code")
```

```
c = Child()
```

```
c.skills()
```

Uses MRO (Method Resolution Order) to decide which parent's method to call first.

(3) Multilevel Inheritance

```
class Grandparent:
```

```
    def greet(self):
```

```
        print("Hello from Grandparent")
```

```
class Parent(Grandparent):
```

```
    def talk(self):
```

```
        print("Hello from Parent")
```

```
class Child(Parent):  
    def play(self):  
        print("Hello from Child")
```

```
c = Child()
```

```
c.greet()
```

```
c.talk()
```

```
c.play()
```

(4) Hierarchical Inheritance

```
class Parent:  
    def show(self):  
        print("Parent property")
```

```
class Child1(Parent):  
    pass
```

```
class Child2(Parent):
```

```
    pass
```

```
obj1 = Child1()
```

```
obj2 = Child2()
```

```
obj1.show()
```

```
obj2.show()
```

(5) Hybrid Inheritance

```
class A:
```

```
    def featureA(self): print("Feature A")
```

```
class B(A):
```

```
    def featureB(self): print("Feature B")
```

```
class C(A):
```

```
    def featureC(self): print("Feature C")
```

```
class D(B, C): # Hybrid
```

```
    def featureD(self): print("Feature D")
```

```
d = D()
```

```
d.featureA()
```

```
d.featureB()
```

```
d.featureC()
```

```
d.featureD()
```

5. Constructor Inheritance (`__init__` Chaining)

When a child class defines its own constructor, the parent's `__init__()` doesn't run automatically — you must explicitly call it using `super()`.

```
class Parent:
```

```
    def __init__(self):  
        print("Parent constructor")
```

```
class Child(Parent):
```

```
    def __init__(self):  
        super().__init__()  
        print("Child constructor")
```

```
obj = Child()
```

Output:

Parent constructor

Child constructor

This process is called constructor chaining.

Real-world analogy:

- Parent sets up “family name” → Child adds their own “first name”.
- Without calling the parent's constructor, setup is incomplete.

6. The `super()` Function

`super()` allows you to access the parent class methods or constructor dynamically — without explicitly naming the parent.

Helps in:

- Constructor chaining
- Method overriding
- Multiple inheritance consistency (via MRO)

Example:

```
class A:
```

```
    def show(self):  
        print("Class A")
```

```
class B(A):
```

```
    def show(self):  
        super().show()  
        print("Class B")
```

```
b = B()
```

```
b.show()
```

Output:

Class A

Class B

7. Method Resolution Order (MRO)

When multiple parents exist, Python must decide which method to execute first.

MRO defines that order.

Check it:

```
print(ClassName.__mro__)
```

Example:

```
class A: pass
```

```
class B(A): pass
```

```
class C(A): pass
```

```
class D(B, C): pass
```

```
print(D.__mro__)
```

Output:

```
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>)
```

Python follows the C3 Linearization algorithm for consistent MRO.

8. Method Overriding

If both parent and child have the same method name, the child method overrides the parent's method.

Example:

```
class Animal:
```

```
def sound(self):  
    print("Animal sound")
```

```
class Dog(Animal):  
    def sound(self): # Overriding  
        print("Bark!")
```

```
d = Dog()  
d.sound()
```

Output:

Bark!

9. Polymorphism (via Inheritance)

Polymorphism means many forms — the same method behaves differently based on the object.

Example:

```
class Bird:  
    def fly(self):  
        print("Birds can fly")
```

```
class Penguin(Bird):  
    def fly(self):  
        print("I cannot fly, I swim")
```

```
for b in [Bird(), Penguin()]:
```

```
    b.fly()
```

Output:

Birds can fly

I cannot fly, I swim

Both share the same method name, but behavior differs → true polymorphism.

10. Abstract Classes (Inheritance with Rules)

When you want all subclasses to follow a common “contract”, use abstract base classes.

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        pass
```

```
class Circle(Shape):
```

```
    def __init__(self, r): self.r = r
```

```
    def area(self):
```

```
        return 3.14 * self.r ** 2
```



```
c = Circle(5)
```

```
print(c.area())
```

Forces subclasses to implement specific methods → strong design control.

Polymorphism in OOPs:

Same Name, Different Behavior”

Polymorphism means *“many forms”*.

In OOP, it allows objects of different classes to be treated as objects of a common superclass — but each object can behave differently when a method is called.

Think of it like this:

“You can use the same method name, but the behavior changes depending on the object that’s using it.”

Real-World Intuition:

Imagine you have a `make_sound()` method.

- A **Dog** barks
- A **Cat** meows
- A **Cow** moos

Even though the method name `make_sound()` is the same, the **action differs** based on the object (animal).

That’s *polymorphism*.

Types of Polymorphism in Python:

1. **Compile-time Polymorphism (Overloading)** – Python doesn't truly support it like Java/C++ because functions can't have multiple definitions with the same name.
But we can **mimic** it using *default arguments* or **args*.
2. **Run-time Polymorphism (Overriding)** – Child class redefines a method of parent class.

Example 1: Function Polymorphism (Same Function, Different Types)

Same function behaving differently depending on type of data

```
print(len("Python"))      # For string -> counts characters
print(len([1, 2, 3, 4])) # For list -> counts elements
print(len({"a": 10, "b": 20})) # For dict -> counts keys
```

Intuition:

`len()` is polymorphic — same name, but it adapts based on the object type.
This is *built-in polymorphism* in Python.

Example 2: Method Overriding (Runtime Polymorphism)

```
class Animal:
    def make_sound(self):
        print("Some generic animal sound")

class Dog(Animal):
```

```
def make_sound(self):  
    print("Woof Woof!")  
  
class Cat(Animal):  
    def make_sound(self):  
        print("Meow Meow!")  
  
# Same method name, different behavior  
animals = [Dog(), Cat(), Animal()]  
  
for animal in animals:  
    animal.make_sound()
```

Output:

Woof Woof!

Meow Meow!

Some generic animal sound

Intuition:

Here the same method name `make_sound()` behaves differently depending on the object type at *runtime*.

See code in the collab notebook

ABSTRACTION — “Hiding Complexity, Showing Essentials”

Abstraction means hiding the implementation details from the user and only exposing necessary functionality.

“When you drive a car, you just press the accelerator — you don’t worry about how the engine mixes air and fuel.”

Real-World Analogy:

You use a **bank ATM**:

- You enter PIN and amount.
- Machine dispenses cash.

You don’t know the internal code, APIs, or data encryption happening inside — that’s *abstraction*.

Why Abstraction?

It:

- Simplifies the interface.
- Hides complex implementation.
- Increases security and maintainability.

How in Python?

Python provides **Abstraction** through the **abc (Abstract Base Class)** module.

You use:

- `from abc import ABC, abstractmethod`
- You can’t instantiate an abstract class.

- Any class inheriting it must implement the abstract methods.